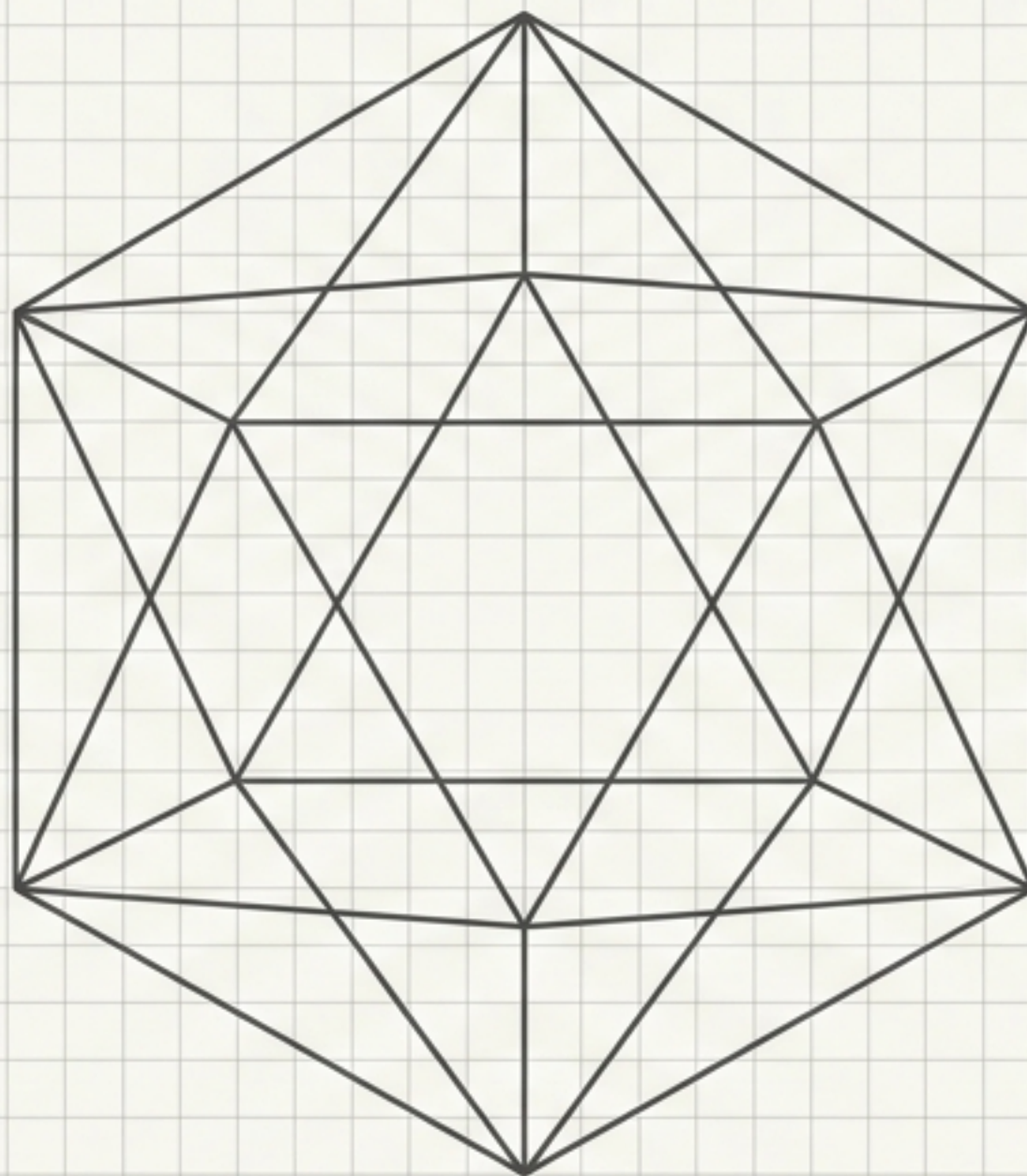


# A Arquitetura do WebGL: Do Dado ao Pixel

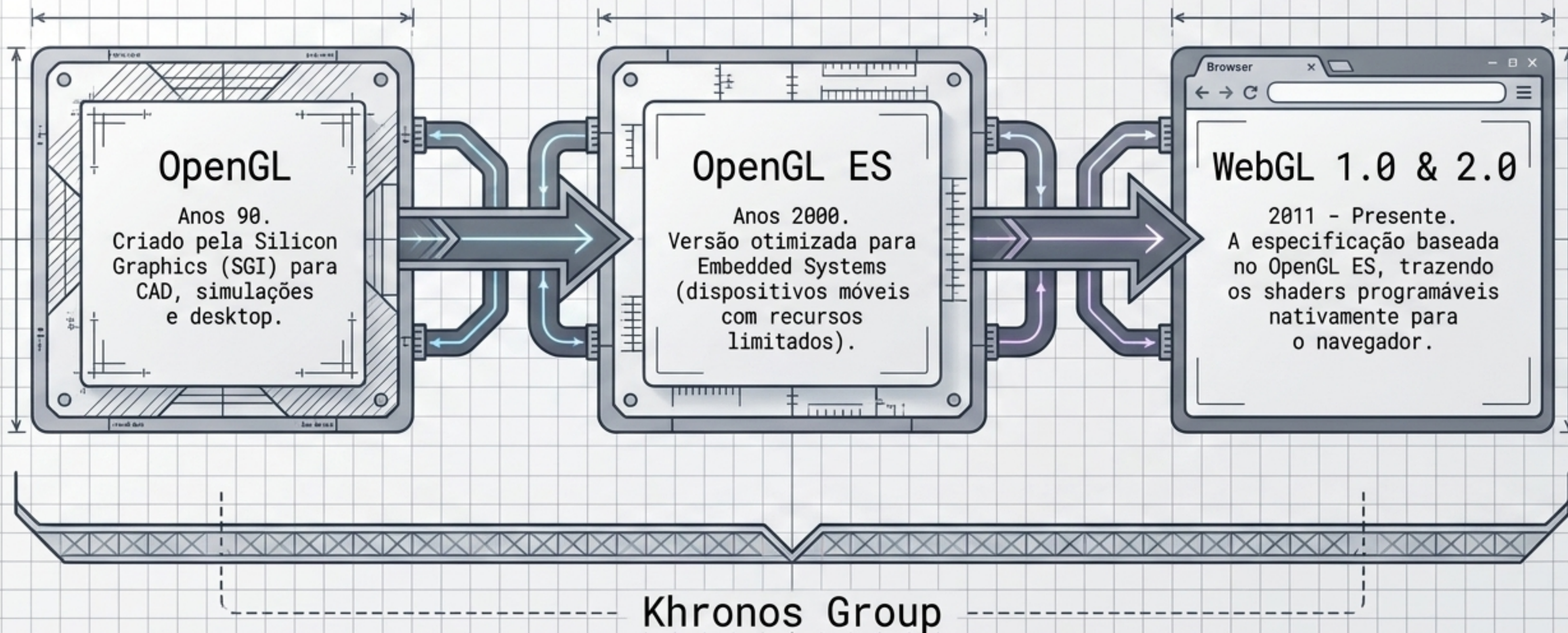
Entendendo o pipeline gráfico, shaders e o futuro da renderização na Web.



# A Ponte Entre o Navegador e a GPU



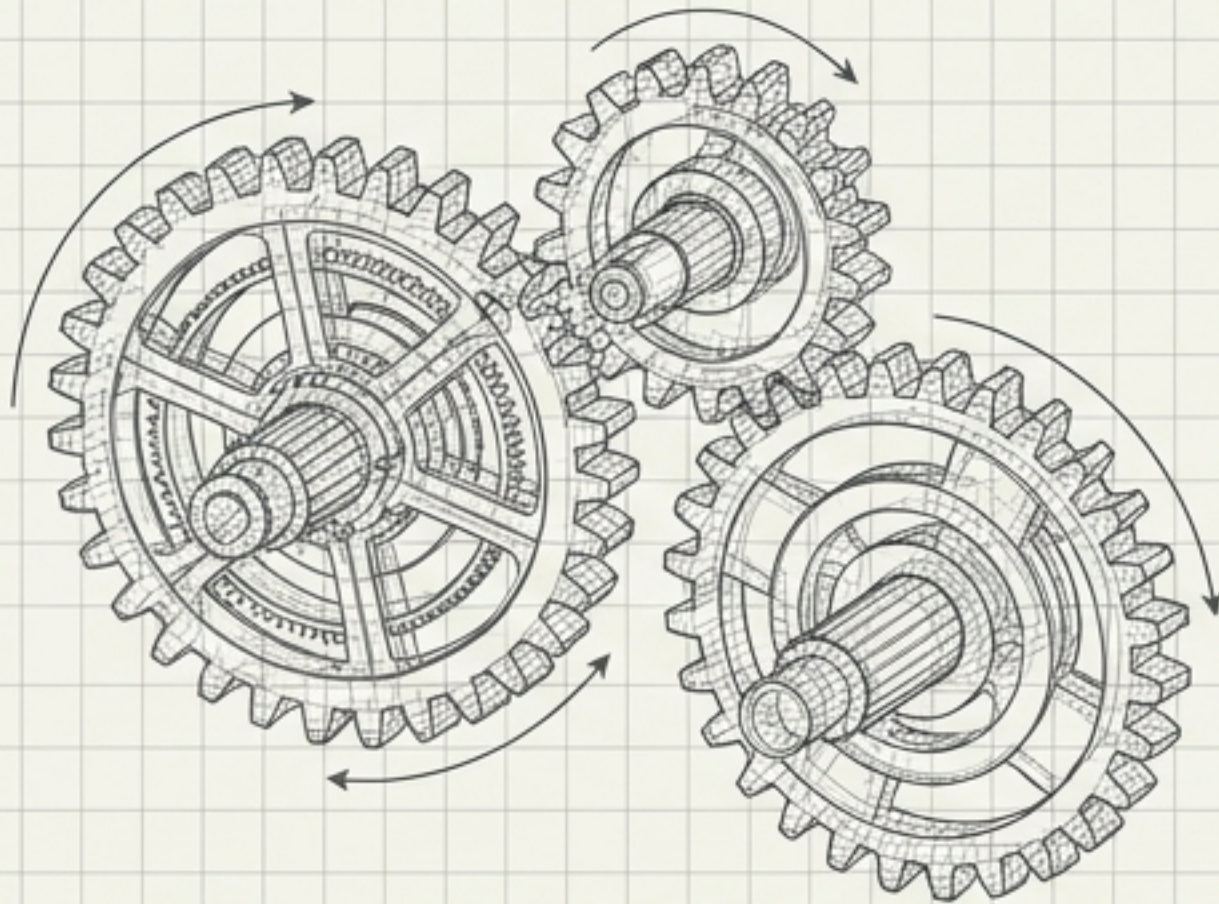
# A Linhagem Tecnológica: Do Desktop ao Navegador



# Por que precisamos da GPU?

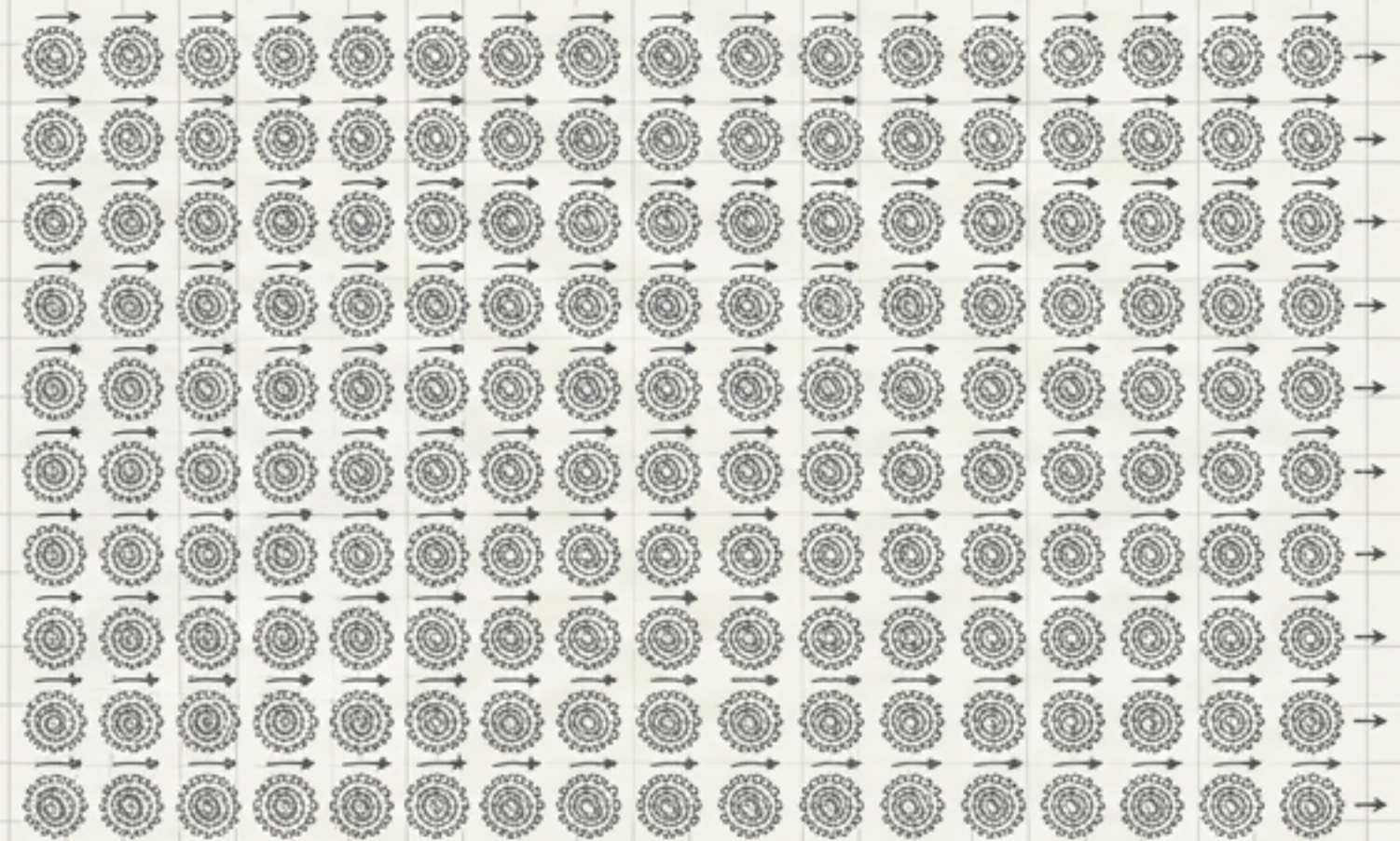
Um monitor comum de 1920×1080 significa calcular cor, luz e profundidade para mais de 2 milhões de pixels, 60 vezes por segundo.

## CPU: Processamento Sequencial Rápido



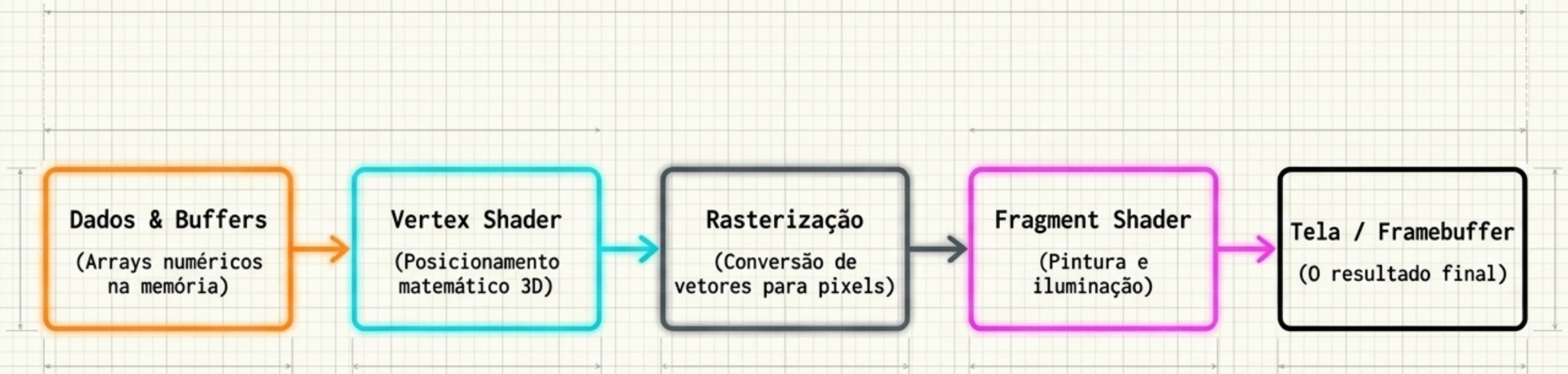
Excelente para a lógica da aplicação,  
ineficiente para renderização em massa.

## GPU: Arquitetura Maciçamente Paralela



Executa cálculos matemáticos simples em  
milhões de fragmentos simultaneamente.

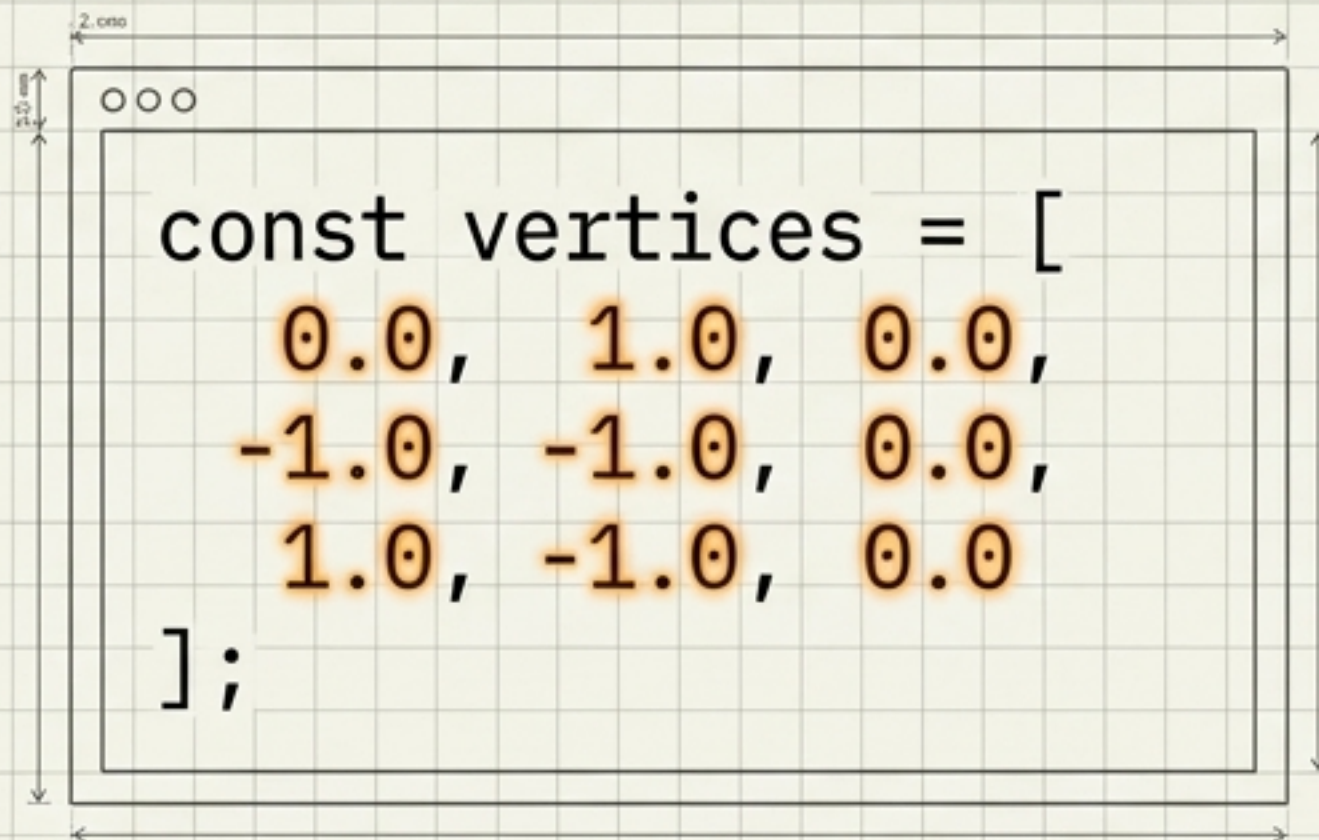
# O Pipeline Gráfico: A Jornada de um Vértice



# A Matéria-Prima: Buffers e Primitivas

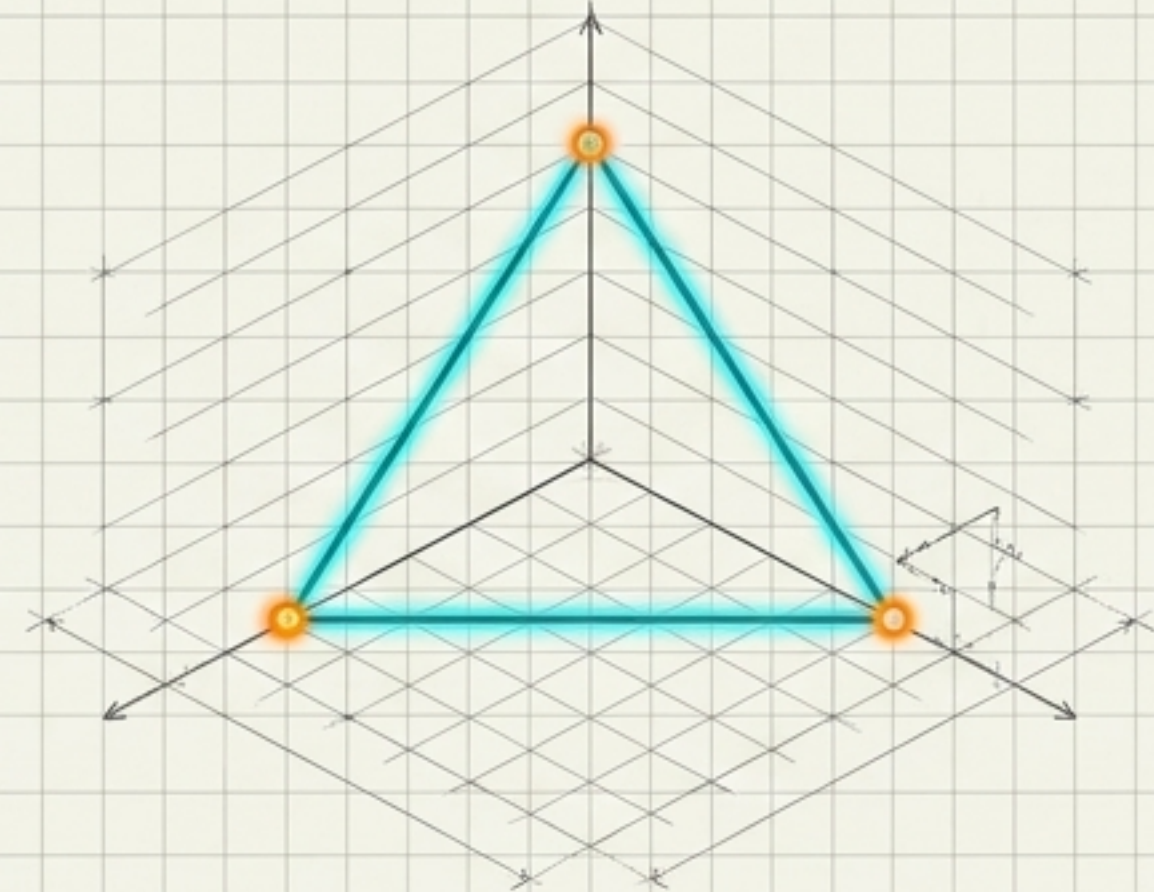
## Buffers: O WebGL só entende números

Os objetos são compostos por matrizes numéricas armazenadas diretamente na memória da GPU através de buffers (ARRAY\_BUFFER).



```
const vertices = [  
    0.0, 1.0, 0.0,  
    -1.0, -1.0, 0.0,  
    1.0, -1.0, 0.0  
];
```

## A Primitiva: O Triângulo



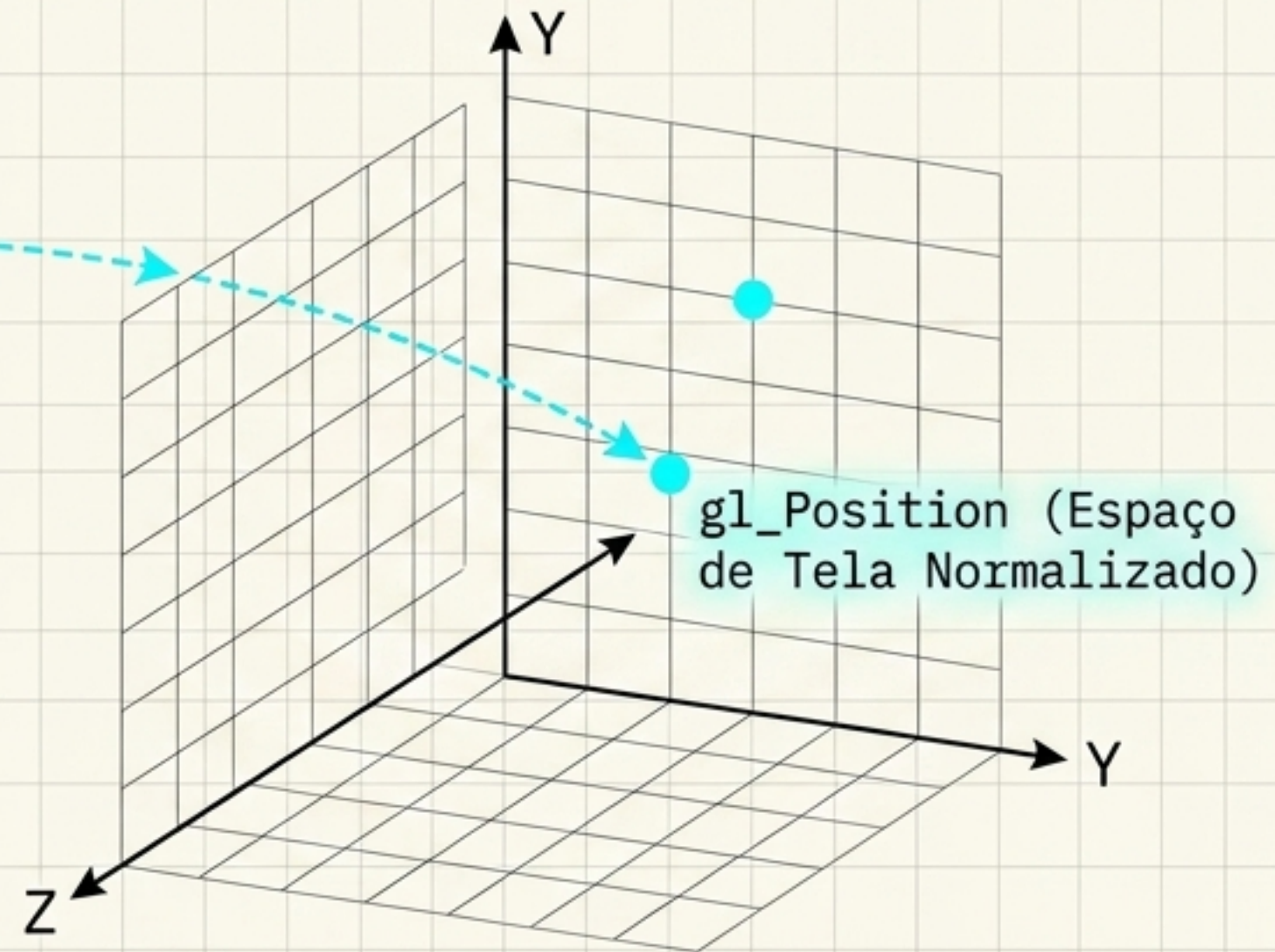
A GPU não recebe cubos ou esferas. Ela constrói universos complexos a partir de primitivas básicas: pontos, linhas e, o mais importante, triângulos.

# Vertex Shader: Esculpindo o Espaço

Um miniprograma executado na GPU para cada vértice individualmente.

```
in vec4 a_position;

void main() {
    gl_Position = a_position;
}
```

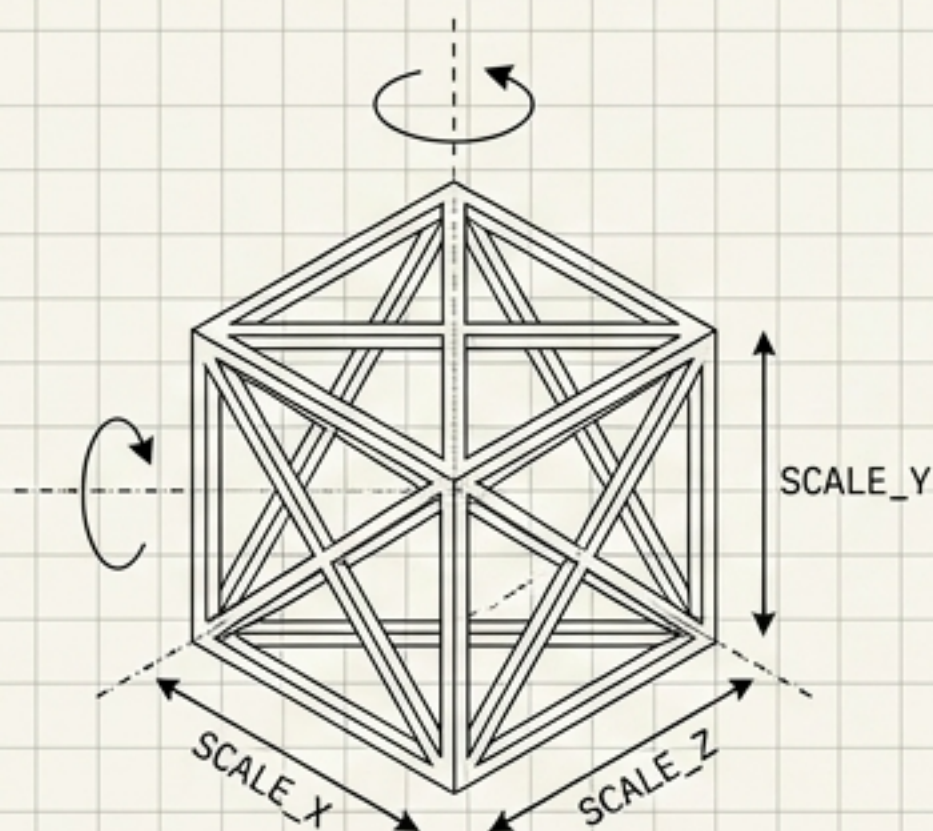


O objetivo é traduzir o modelo do seu próprio sistema de coordenadas para o espaço de tela.

# A Matemática do Posicionamento: Matrizes MVP

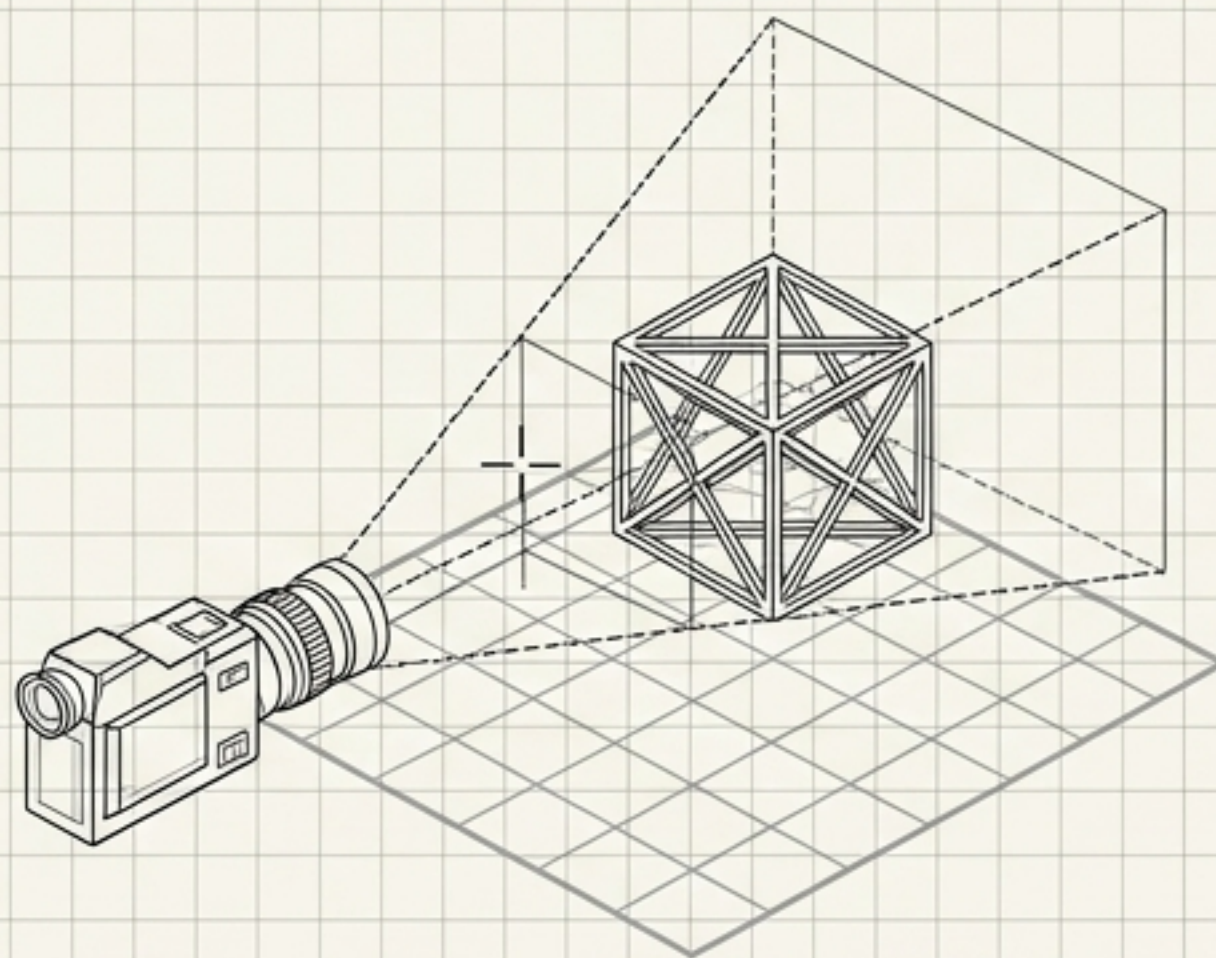
## Passo 1: Model (Modelo)

Transforma o objeto localmente.



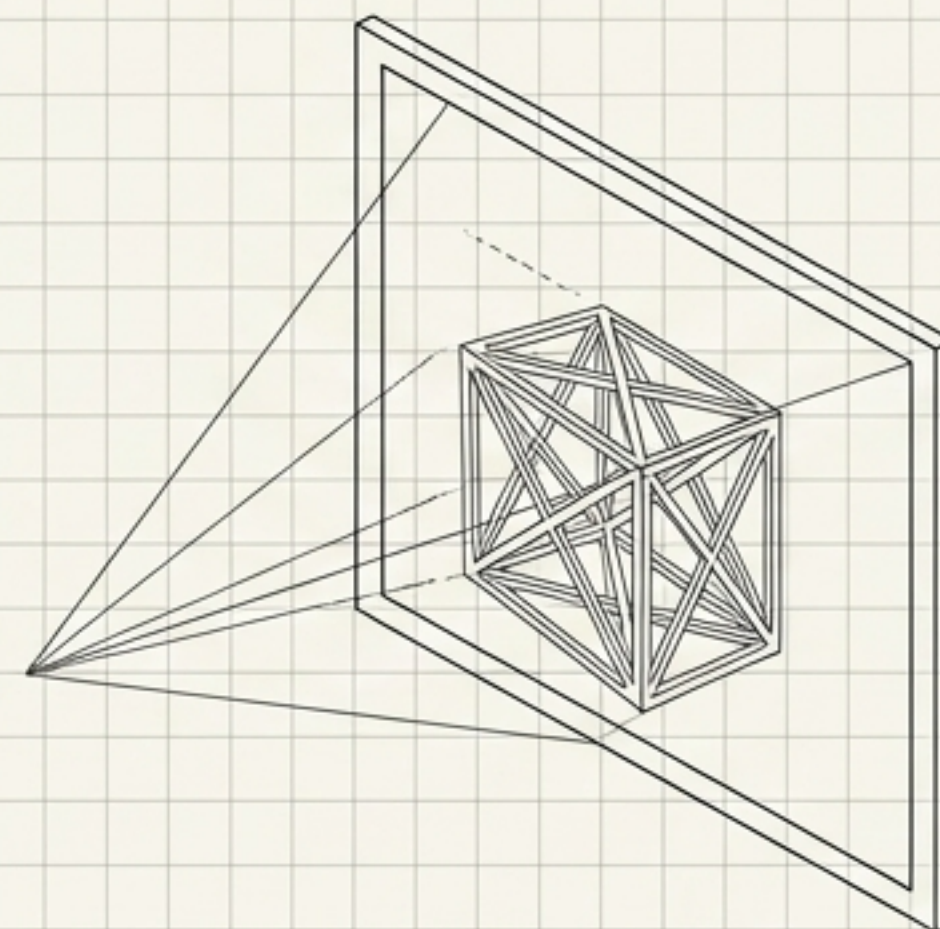
## Passo 2: View (Câmera)

Define o ponto de observação.



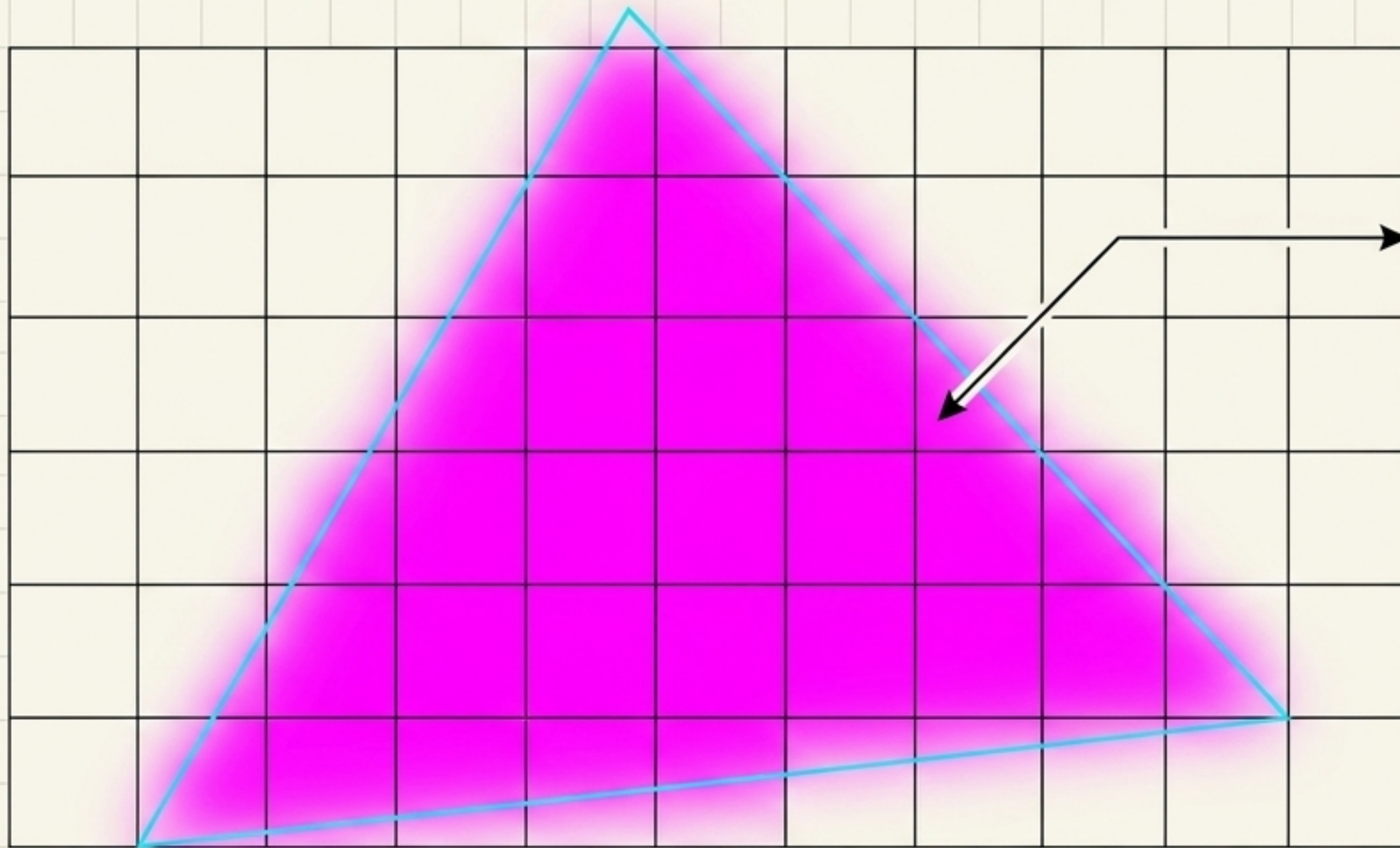
## Passo 3: Projection (Projeção)

Achata a cena tridimensional para uma tela 2D.



# Rasterização: A Conversão Discreta

A ponte crítica entre os vértices matemáticos e a tela física.

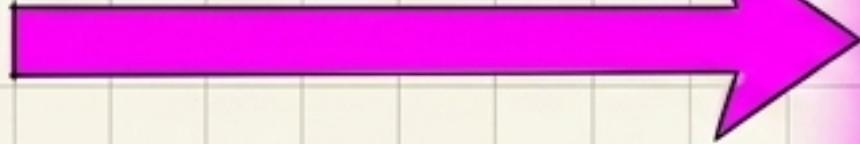


A GPU analisa a malha da tela e descobre exatamente quais pixels pertencem ao interior do triângulo. Cada pixel identificado gera um fragmento.

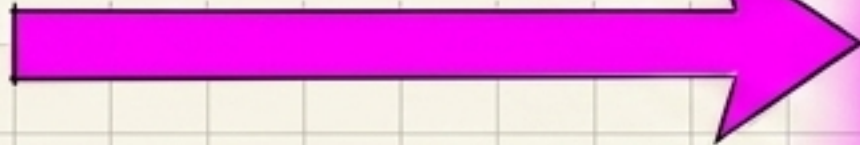
# Fragment Shader: Pintando a Realidade

Um miniprograma executado para cada fragmento (pixel) gerado na rasterização. Seu único objetivo é determinar a cor final.

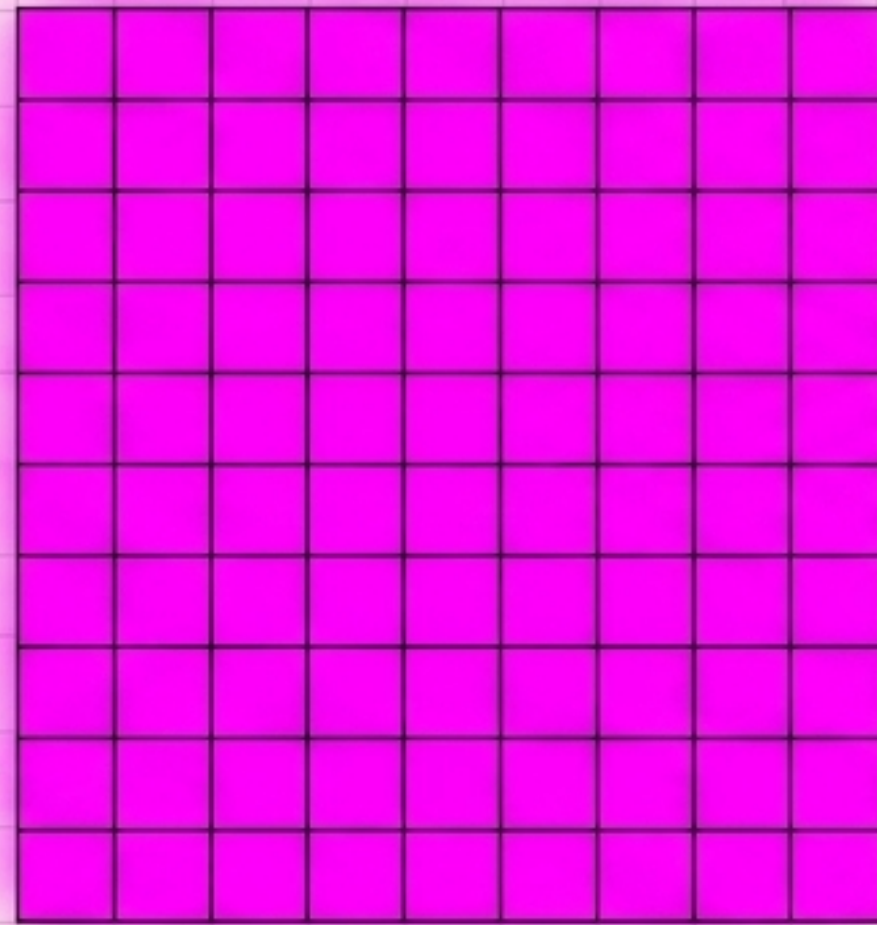
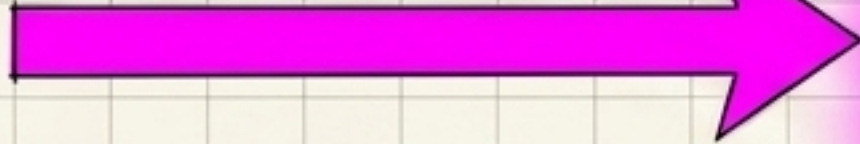
Cor Interpolada (Varying)



Dados da Textura (UV)



Dados de Iluminação

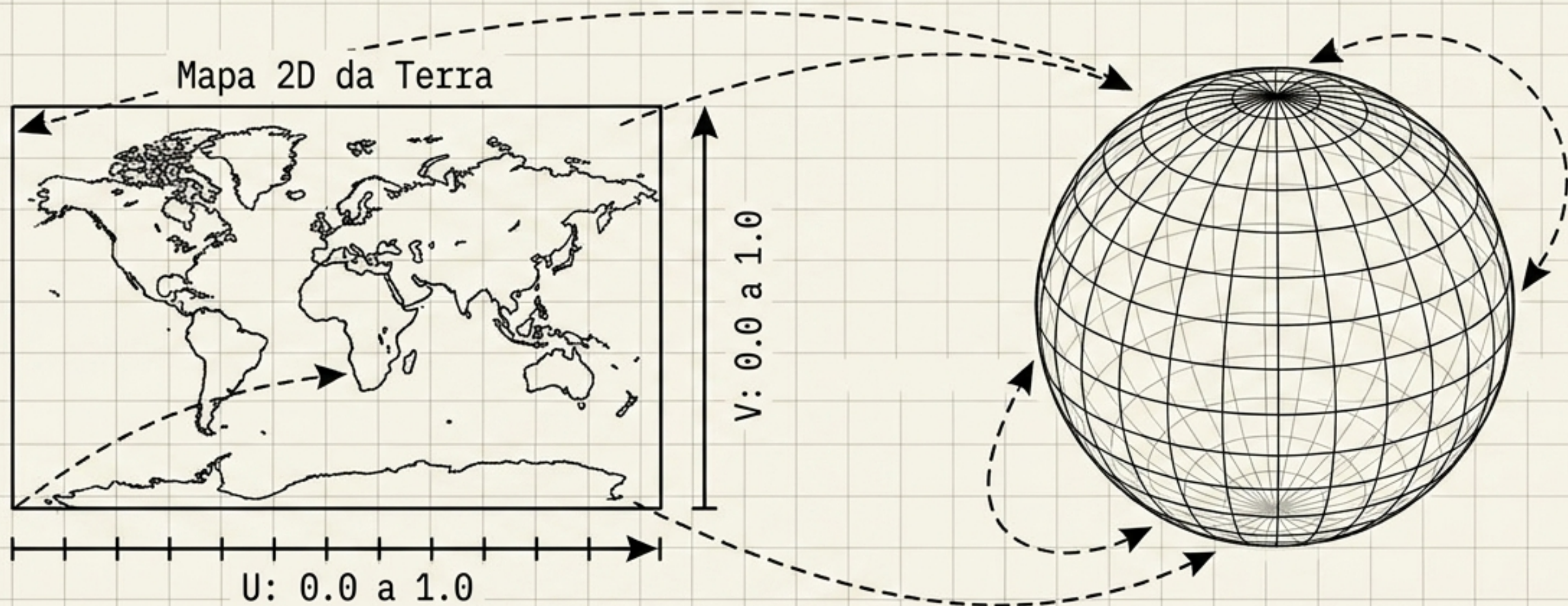


```
outColor = vec4(1.0, 0.0, 0.0, 1.0);  
// Vermelho opaco
```

Interpolação: O WebGL mistura suavemente os valores matemáticos no espaço entre os vértices, criando gradientes perfeitos para os fragmentos processados.

# Texturas e Coordenadas UV: 0 Envelopamento

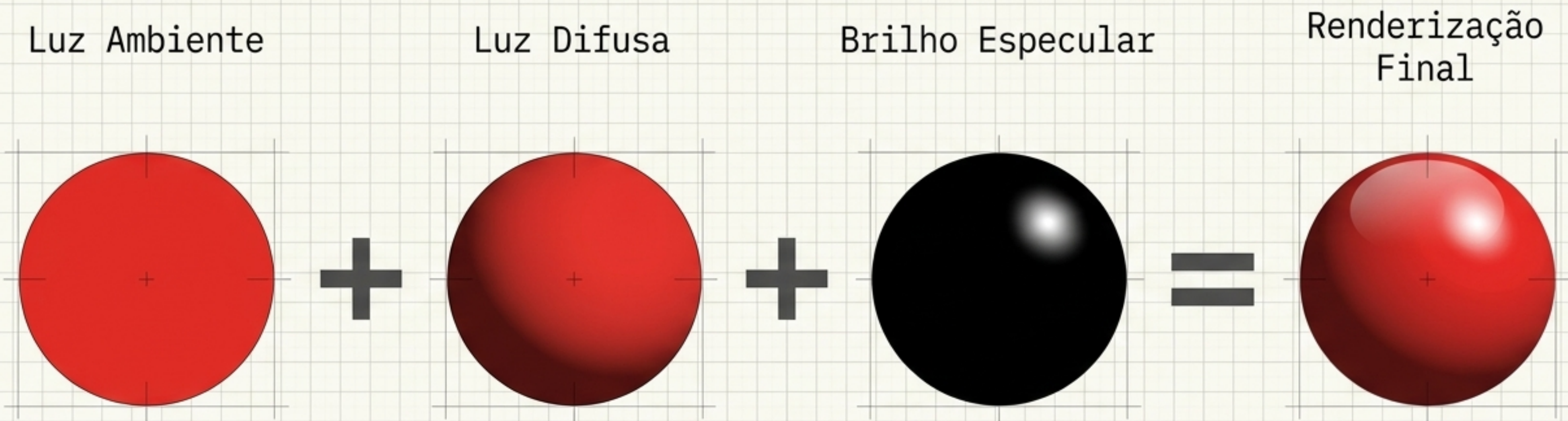
Como o WebGL sabe qual parte de uma imagem 2D deve ser aplicada a um modelo 3D?



Otimização (MIPMAPS): A GPU gera versões menores da mesma textura para objetos distantes, economizando processamento e evitando ruídos visuais.

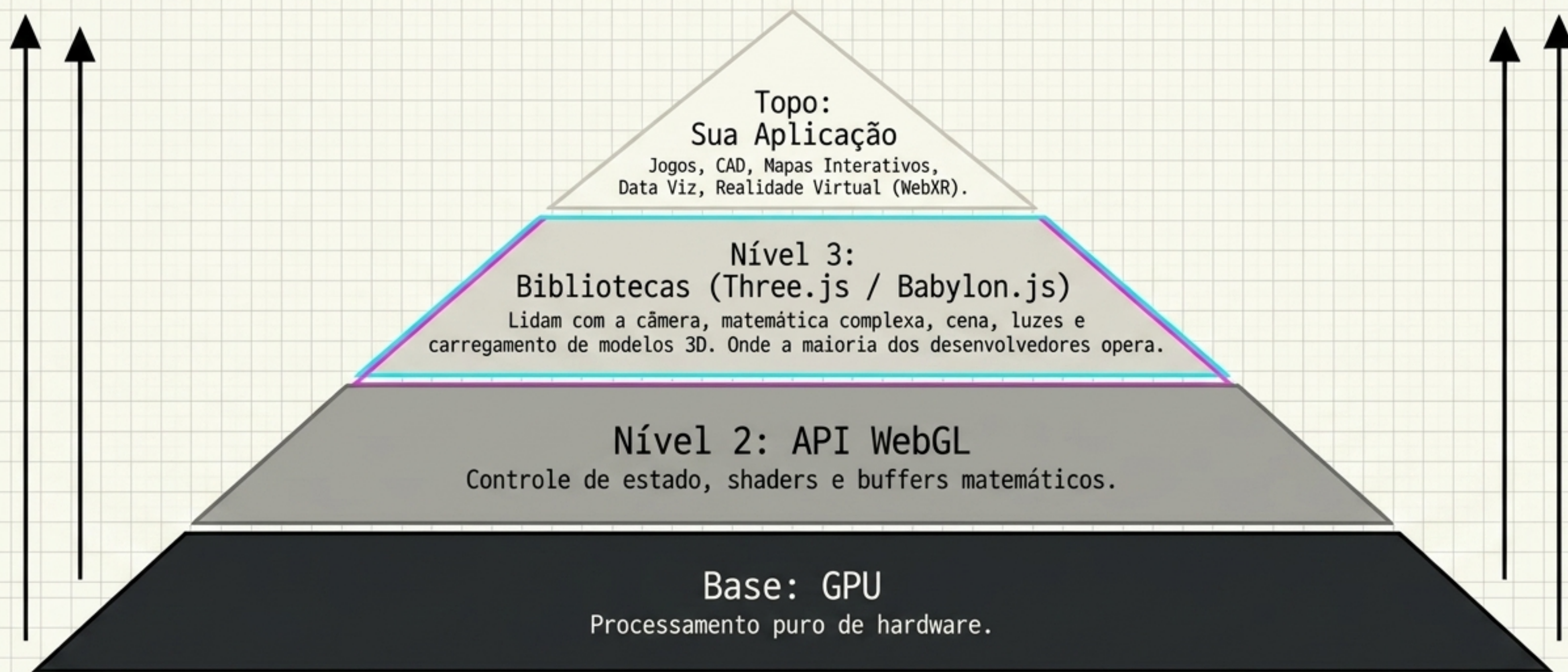
# Iluminação e Profundidade (Modelo de Phong)

Dando volume aos pixels usando Normais (vetores perpendiculares à superfície).



# O Ecossistema: O WebGL Não é uma Engine

O WebGL é uma API de baixo nível. O desenvolvimento moderno depende de abstrações.



# O Futuro da Renderização: WebGL vs. WebGPU

| Característica        | WebGL                                  | WebGPU                                                           |
|-----------------------|----------------------------------------|------------------------------------------------------------------|
| Origem / Inspiração   | OpenGL ES (Antigo, consolidado)        | Vulkan, Metal, Direct3D 12 (Moderno)                             |
| Arquitetura           | Baseado em Máquina de Estado           | Baseado em Objetos e Pipelines                                   |
| Foco de Processamento | Estritamente Gráficos (Shaders)        | Gráficos Avançados e Computação Geral (Física, Machine Learning) |
| Status Atual          | Suporte universal, ecossistema gigante | O futuro da altíssima performance na Web                         |

O WebGL não está morto. Ele continua sendo a fundação de compatibilidade universal, enquanto o WebGPU assume os limites extremos de desempenho.

## Síntese da Jornada

“WebGL é a arte e a engenharia de transformar dados brutos em geometria, projetá-la no espaço matemático, cortá-la em fragmentos discretos e pintá-la com luz.”

Dados → Buffers → Vértices → Vertex Shader → Rasterização → Fragment Shader → Tela